

Einladen

- Hallo, ich bin Nathan Corral.
 - Ich wollte erstmal du/Sie für die Gelegenheit heute hier zu sein bedanke
 - Ich freue mich hier zu sein.
- Ein paar Hintergrundinformationen zu mir:
 - Ich habe 2017 meinen Bachelor in Computer Engineering an der University of Illinois abgeschlossen.
 - Danach bin ich nach Colorado gezogen, wo ich drei Jahre lang in Embedded Systems und als Softwareentwickler gearbeitet habe.
 - Im Jahr 2020 bin ich nach Deutschland gezogen, um einen Master in Intelligent Systems zu absolvieren.
 - Dadurch konnte ich mein Interesse an Robotik, das ich während meines Bachelors entwickelt habe, mit meiner damaligen Interesse für künstliche Intelligenz und deren Potenzial kombinieren.
- Ich habe jetzt eine breite Palette an Erfahrungen gesammelt – von Hardwareprogrammierung über Middleware-Entwicklung (z. B. Aufbau von Pipelines zur Visualisierung von Sensordaten) bis hin zur Forschung in moderner Computer Vision.
- Diese Stelle bei Arobo weckt großes Interesse bei mir, da sie perfekt zu meinem Hintergrund passt
 - Ich freue mich darauf, mehr über Ihr Team zu erfahren und darüber, wie ich gemeinsam mit Ihrem Team innovative Lösungen entwickeln kann.

Projekte

ROS 2 CV

- Dies war ein Projekt, das ich aus persönlichem Interesse begonnen habe, um mich besser mit ROS 2 und den Hugging Face-Bibliotheken vertraut zu machen (was zu meinen Karrierezielen passt, KI und Robotik zu kombinieren).
 - Falls Sie damit nicht vertraut sind: Hugging Face hostet verschiedene State-of-the-Art KI-Modelle und Datensätze und bietet zahlreiche Bibliotheken, die z.B. das Training oder die Inferenz dieser Modelle ermöglichen
 - (Und Sie hat, zumindest von ROS gehört haben) Und ROS ist eine Middleware für Robotik, die asynchrone, modulare Kommunikation ermöglicht.
- Wie in ROS-Projekten üblich, habe ich eine Pipeline aus drei Stufen entworfen:
 - Die erste Stufe dieser Pipeline würde ein Bild aus entweder: dem Live-Kamera-Feed; früheren Aufnahmen; oder einem aus einem Hugging Face-Datensatz.
 - Dies ist sehr nützlich, da es eine Möglichkeit bietet, die Funktionalität des Modells in der Produktion zu testen.
 - In der mittleren Stufe würden diese Bilder an die verschiedenen Computer-Vision-Modelle gesendet, die Erkennungen vornehmen.

- Was ich als Beispiele für Erkennungen verwendet habe, waren Bounding Boxes und Segmentierungsmasken.
- Zusätzlich zur Bearbeitung der Eingaben würden die Objektklassifikations-Indizes in einer globalen Tabelle neu indiziert.
 - Dies musste gemacht werden, da die Modelle möglicherweise auf verschiedenen Datensätzen trainiert wurden.
- In der letzten Stufe erfolgte die Visualisierung der Ergebnisse.
 - Diese Stufe empfing sowohl das Bild als auch die vorgenommenen Erkennungen, die möglicherweise eine zeitliche Verschiebung aufwiesen, und zeigte dann die Ergebnisse als Live-Matplotlib-Animation an.

Meisterarbeit

Der Titel meiner Masterarbeit war: „**Stochastic Transformers for Prediction of Multiple Futures**“.

In meiner Arbeit habe ich mich darauf konzentriert, die Vorhersage von Videos zu verbessern. Ein häufiges Problem bei der Videovorhersage ist, dass kleine Fehler, wie ein vorhergesagtes Bild, das nur um einen Pixel verschoben ist, während des Trainings zu starken Bestrafungen führen können. Um dieses Problem zu lösen, habe ich eine Methode verwendet, die von früheren Arbeiten zur Videovorhersage und von Architektur aus dem Natural Language Processing inspiriert wurde. Statt nur eine einzige Vorhersage zu machen, lernt die Methode eine Verteilung über mehrere mögliche Zukünfte.

Der Ansatz basiert auf einer Architektur mit zwei Zweigen. Ein Zweig durfte während des Trainings „in die Zukunft sehen“, indem er das tatsächliche Ground Truth nächste Bild als Input erhielt. Der andere Zweig arbeitete normal ohne diese Information -- nur mit dem aktuell Bild. Der „in die Zukunft sehende“ Zweig erzeugte eine Probe aus seiner Verteilung. Diese Probe wurde dann mit dem aktuellen Bild kombiniert und an einen dritten gemeinsamen Teil der Architektur weitergegeben, um das nächste Bild im Video vorherzusagen. Dieser Zweig, der in die Zukunft sehen konnte, wurde nur im Training genutzt und danach entfernt.

Für das Training haben wir zwei Loss-Funktionen verwendet:

1. Ein Pixel-zu-Pixel-Vergleich zwischen der Vorhersage und dem Ground-Truth-Bild, um wichtige Bildmerkmale wie Formen und Texturen zu lernen.
2. Die KL-Divergenz, um die Verteilungen der beiden Zweige anzugleichen. Dadurch konnte der Zweig ohne Zukunftsinformationen sinnvolle Vorhersagen über mögliche Zukünfte lernen.

Diese Methode hat nicht nur die Videovorhersage verbessert, sondern konnte auch auf andere zeitabhängige Aufgaben angewendet werden – zum Beispiel auf die Generation of Human Poses. Durch diese Arbeit habe ich viel über zeitabhängige Daten, Computer Vision und moderne Deep-Learning-Techniken (wie Transformer-basierte Architekturen) gelernt.

Question: How would you apply this, to overcome a simulation to real gap experienced by robots trained only on data from a simulation?

- When training with a Simulation, you have complete control over robots actions, while, in a dataset, all decisions a robot have already been made
- Which can cause problems, if you are training an end-to-end system, where actions are a direct product of the sensors.

- And so, a possible application of this thesis, would be that, in the same way multiple futures are represented by a distribution and then sampled, actions could also be treated this way.
- In this scenario, real datasets could be used to as part of the end-to-end systems with learning of an action distribution.

ROS 2 Whisper

- In diesem Project habe ich auf einer bestehenden Arbeit aufgebaut, die eine C++-Implementierung von Open AI's Whisper als ROS 2 Action Server integrierte, um Audio-Transkription durchzuführen
- Meine Erweiterung bestand darin, eine kontinuierliche Streaming-Transkription hinzufügen
- Zuvor erforderte die Trankription ein festes Zeitfenster und wurde erst nach Ablauf dieses Fenster ausgeführt -- was zu möglichen Wortabbrüchen und einem Verlust des Kontexts führte.
- Dies habe ich durch die Verwendung eines Ringpuffers für das Audio erreicht -- der zum Beispiel immer die letzten 10 Sekunden der Audiodaten als Wellenform hielt.
- Dann wurden in regelmäßigen Abständen, zum Beispiel jede Sekunde, alle Audiodaten im Ringpuffer transkribiert.
- In diesem Beispiel würde das Ergebnis etwa 9 Sekunden zuvor gesehener Text und 1 Sekunde neuer Text umfassen
- In einem separaten Thread habe ich einem effizienten Algorithmus für Substring-Matching verwendet, um diese eingehenden Text mit dem Bereits bestehenden Transkript abzugleichen
- Dadurch konnte ich die Vertrauenswert für erneut auftretende Wörter aktualisieren und den neu transkribierten Text hinzufügen
- Mein Beitrag führte zu Veröffentlichung der Version 1.4 und dazu, dass ich ein Maintainer des Project wurde
- Zusätzlich gelang es mir, diese Lösung auf einem Nvidia Jetson Orin NX auszuführen, was meine Fähigkeit zeigte, auf realen Geräte zu arbeiten und einzusetzen.

Job Aufgaben

SLAM / Kalman Filters

- How I have used Particle Fitlers (which is based off the Kalman Filter) was that, given a dataset of noisy object detections and movements of the robot, perform localization.
- This was done in as part of a class assignment, and is an integral component is SLAM.
- I am very familiar with theoretical basics, and am a strong programmer able to implement them.

Sensors

- Aqronos LiDAR
- Humanoid Robots Lab RBGD

Harware Programming

- Head Rush Tech

Fragen

1. Forschung oder ähnlich Projekt

- Ich interessiere mich für die Balance zwischen täglichen Aufgaben und langfristigen oder forschungsorientierten Projekten bei Arobo.
- Gibt es die Möglichkeit, an zukunftsorientierten Projekten teilzunehmen, bei denen ich meine Fähigkeit, selbstständig zu arbeiten einbringen kann?

2. Ubuntu

- Aus Neugier, welches Betriebssystem und welche Desktop-Umgebung verwenden Sie hier üblicherweise? Zum Beispiel, ist es Ubuntu mit KDE, Cinnamon, GNOME oder etwas ganz anderes?

3. ROS (2)

- Welches Middleware verwenden Sie für Ihre Roboter?
 - Alternativen zu ROS z.B. YARP, OROCOS, Zenoh
- Und was für Simulators?

4. C++

- In der Stellenbeschreibung werden C++ Kenntnisse zur hardwarenahen Programmierung erwähnt.
- Welche Version von C++ wird verwendet und auf welcher Art von Hardware

5. Hardware Sensors

- Welche Sensoren verwenden Sie bei Ihrem rs3-Modell?

6. Arbeitsgruppe

- Wie groß ist die Team, dem ich angehören würde?

7. Nächsten Schritten im Einstellungsverfahren

- Was sind die

8. Senior Team Member

- Wenn technische Herausforderungen auftreten, gibt es einen festen Ansprechpartner oder einen erfahrenen Ingenieur, den das Team typischerweise zur Unterstützung bei der Problemlösung konsultiert?

9. Hunde

- Aus allgemeinem Interesse, sind Haustiere im Büro erlaubt

10. Remote-Arbeit

- Da ein Großteil der Arbeit, wie Sie erwähnt haben, die direkte Arbeit am Roboter beinhaltet, gibt es dennoch Flexibilität für Remote-Arbeit?

Zusätzliche Projekte

Humanoid Robots Lab

- Eine der Aufgaben, die mir während meiner Tätigkeit im Humanoid Robots Lab übertragen wurde, war die Lokalisierung einer Person in 3D-Koordinaten mit Hilfe einer RGBD-Kamera.
- Zunächst hielt ich ein Seminar über Techniken der Computer Vision, die Teillösungen bieten konnten – Identifikation von Pixeln im RGB-Bild, die einer Person entsprechen.
- Nach Diskussionen mit dem Team entschieden wir uns für die Verwendung eines YOLO-Netzwerks ("You Only Look Once"), das Objekterkennung in Kamerabildern schnell durchführt.
- Nach der Implementierung als ROS-Node konnten wir eine Begrenzungsbox um jede Person in einer Szene identifizieren, wobei wir einen Konfidenzschwellenwert als Hyperparameter zur Filterung von Erkennungen verwendeten.
- Dies war jedoch nur eine Teillösung, da wir eigentlich Folgendes benötigten:
 1. Die Distanz von der Person zur Kamera
 2. Den relativen Winkel in der Horizontalebene von der Kamera zur Person
- Um die Distanz von der Person zur Kamera zu bestimmen, verwendeten wir den Tiefenkanal des Kamerabildes.
- Wir begannen damit, alle Tiefenwerte der Pixel, die Teil der Erkennung waren, in Bins zu gruppieren und dann ein spezifisches Quantil auszuwählen, das wir durch empirische Analyse ermittelten.
- Zur Bestimmung des Winkels verwendeten wir das mittlere Pixel der Begrenzungsbox und berechneten es basierend auf dem Sichtfeld der Kamera.

- Dies war erfolgreich und wurde als Teil eines Pfadplanungsalgorithmus verwendet, der auf einem echten Roboter ausgeführt wurde.

Aqronos

- Während meiner Zeit bei Aqronos war ich verantwortlich für die Entwicklung der Visualisierungssoftware für das LiDAR-Produkt des Unternehmens.
- Mit C++ und ROS habe ich eine Pipeline erstellt, die folgende Aufgaben umfasst:
 - das Zusammenfügen von über UDP eingelesenen Punkten zu einem Frame;
 - die Durchführung einer strukturierten Filterung der Punkte im Frame;
 - sowie die Konvertierung des Frames in eine Punktwolke und das Anwenden von Filtern aus der C++ Point Cloud Library.
- Mithilfe eines Qt-Backends (rqt_reconfigure) habe ich eine GUI implementiert, die es dem Benutzer ermöglicht, Parameter der Filteroperationen dynamisch zu steuern.
- In Zusammenarbeit mit dem Embedded-Systems-Team habe ich mit einem REST-Server auf dem LiDAR interagiert, um mechanische Parameter wie den Scanwinkel und die Aktualisierungsrate des Scans dynamisch einzustellen.

Head Rush Tech

- In diesem Projekt habe ich die Firmware für eine kundenspezifische Leiterplatte (PCB) programmiert, die in einem Proof-of-Concept Prototype verwendet wurde
 - Es gibt ein Fehler auf mein Lebenslauf, was ich habe Ihnen geschickt, dass sagte was folgte dieses Proof-of-Concept. (dass ich habe korrigiert)
 - Aber ich habe nur das Firmware des Prototyps programmiert, und ich war während der Fortführung der Prozesse nicht beschäftigt.
 - Wenn Sie möchten, kann ich die aktualisierte Version meines Lebenslauf schicken.
- Ziel des Prototyps war es, eine neue Funktion für das bestehende Selbstsicherungs-System zu entwickeln, indem eine softwaregesteuerte Bremse hinzugefügt wurde.
- Diese Bremse sollte bei einem Sturz des Kletterers aktiviert werden, um ihn in der Luft zu stoppen und zu halten, anstatt ihn wie im Normalbetriebs langsam zu Boden zu lassen.
- Wie es funktioniert war...
 - Das System verwendete einen Zahnsensor, der in einem Gerät am oberen Ende der Wand angebracht war.
- Dieses Gerät war über eine Spule mit dem Kletterer verbunden.
- Beim Auf-oder Absteigen des Kletterers wickelte sich die Spule ab, und der Zahnsensor erfasste die Bewegung.
- Das war das Kontext, und ..
 - Ich habe einen ATMEGA- Mikrocontroller programmiert, um Interrupts auszulösen, wenn der Zahnsensor aktiviert wurde
- Zur Verbesserung der Signalstabilität habe ich Software-Debouncing implmentiert, um Störsignale zu eliminieren.

- Ich habe eine Zustandsmaschine entworfen und implementiert, um drei Hauptaufgaben zu erfüllen
 - 1. Erkennen eines Sturzes durch Analyse der Abwärtsbewegung und Beschleunigung
 - 2. Aktivieren der Bremse bei einem Sturz mithilfe eines kontrollierten PWM-Signals,
 - um ein plötzliches Anhalten zu vermeiden
 - 3. Lösen der Bremse, sobald der Kletterer wieder aufstieg
- Und für die letzte Funktion, was ich habe programmiert,
 - Da das System zwei Geräte umfasste -- eines am oberen Ende der Wand und eines am Kletterer -- habe ich die RS485 Kommunikation zwischen ihnen über USART (serielle Kommunikation) implementiert.
- Das Prototyp war erfolgreich, und wir haben die Feldtests abgeschlossen.
- Anschließend habe ich den Code und die umfassende Dokumentation für das Projekt übergeben.

Humanoid Robots Lab

- Eine der Aufgaben, die mir während meiner Tätigkeit im Humanoid Robots Lab übertragen wurde, war die Lokalisierung einer Person in 3D-Koordinaten mit Hilfe einer RGBD-Kamera.
- Zunächst hielt ich ein Seminar über Techniken der Computer Vision, die Teillösungen bieten konnten – Identifikation von Pixeln im RGB-Bild, die einer Person entsprechen.
- Nach Diskussionen mit dem Team entschieden wir uns für die Verwendung eines YOLO-Netzwerks ("You Only Look Once"), das Objekterkennung in Kamerabildern schnell durchführt.
- Nach der Implementierung als ROS-Node konnten wir eine Begrenzungsbox um jede Person in einer Szene identifizieren, wobei wir einen Confidence Threshold als Hyperparameter zur Filterung von Erkennungen verwendeten.
- Dies war jedoch nur eine Teillösung, da wir eigentlich Folgendes benötigten:
 1. Die Distanz von der Person zur Kamera
 2. Den relativen Winkel in der Horizontalebene von der Kamera zur Person
- Um die Distanz von der Person zur Kamera zu bestimmen, verwendeten wir den Tiefenkanal des Kamerabildes.
- Wir begannen damit, alle Tiefenwerte der Pixel, die Teil der Erkennung waren, in Bins zu gruppieren und dann ein spezifisches Quantil auszuwählen, das wir durch empirische Analyse ermittelten.
- Zur Bestimmung des Winkels verwendeten wir das mittlere Pixel der Begrenzungsbox und berechneten es basierend auf dem Sichtfeld der Kamera.
- Dies war erfolgreich und wurde als Teil eines Path Planning Algorithmus verwendet,
- Und zum Schluss habe ich auch viele der auf einem echten Roboter ausgeführt wurde.